Lab 1: Link Layer

WT1

The filter is `eth.src == 6a:15:0a:ba:9b:7c`.

1. There are **827** frames in the filtered results.
2. The destination MAC address of the third filtered frame is `ff:ff:ff:ff:ff:ff`. This is the Ethernet broadcast address, addressing all hosts in the same broadcast domain.
3. The byte at zero-based offset 71 is **`0x15`**.

Code arrangement

- **PT1:** `src/device.c` implements device enumeration, registration, lookup, and cleanup. `src/include/device.h` declares the interfaces.
- **PT2:** `src/packetio.c` implements Ethernet II frame transmission, reception, and callback registration. `src/include/packetio.h` declares the interfaces.
- `src/apps/lab1_demo.c` provides the `list`, `send`, and `recv` commands used for CP1 and CP2.

CP1

Running `lab1_demo list` produces the output below. The program detects the host interface `enp3s0`, loopback `lo`, and other capture devices through libpcap.

```
(base) ubuntu@my-lab:~/labs/netstack-lab$ ./lab1/build/lab1_demo list
enp3s0
any : Pseudo-device that captures on all interfaces
lo
bluetooth-monitor : Bluetooth Linux Monitor
nflog : Linux netfilter log (NFLOG) interface
nfqueue : Linux netfilter queue (NFQUEUE) interface
dbus-system : D-Bus system bus
dbus-session : D-Bus session bus
```

Figure 1: Network interface enumeration.

CP2

Two network namespaces, `l1ns1` and `l1ns2`, are connected by a veth pair: `l1veth1-2` and `l1veth2-1`. The sender injects a frame in `l1ns1`; the custom receiver and tcpdump capture it in `l1ns2`. The commands and their output are shown below.

```
(base) ubuntu@my-lab:~/labs/netstack-lab$ sudo ip netns exec l1ns1 ~/labs/netstack-lab/lab1/build/lab1_de
mo send l1veth1-2
```

Figure 2: Sending a frame with the custom program.

```
(base) ubuntu@my-lab:~/labs/netstack-lab$ sudo ip netns exec l1ns2 ~/labs/netstack-lab/lab1/build/lab1_de
mo recv l1veth2-1
device=0 length=60
ff ff ff ff ff ff 16 05 27 4c 93 c9 88 b5 68 65
6c 6c 6f 20 66 72 6f 6d 20 6c 61 62 31 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00
```

Figure 3: The custom receiver captures and prints the frame.

```
(base) ubuntu@my-lab:~/labs/netstack-lab$ sudo ip netns exec l1ns2 tcpdump -i l1veth2-1 -e -XX 'ether pro
to 0x88b5'
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on l1veth2-1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C12:15:08.445038 16:05:27:4c:93:c9 (oui Unknown) > Broadcast, ethertype Unknown (0x88b5), length 60:
        0x0000:  ffff ffff ffff 1605 274c 93c9 88b5 6865  ........'L....he
        0x0010:  6c6c 6f20 6672 6f6d 206c 6162 3100 0000  llo.from.lab1...
        0x0020:  0000 0000 0000 0000 0000 0000 0000 0000  ................
        0x0030:  0000 0000 0000 0000 0000 0000            ............

1 packet captured
1 packet received by filter
0 packets dropped by kernel
```

Figure 4: tcpdump independently captures the same frame.

Both captures show a 60-byte frame with destination `ff:ff:ff:ff:ff:ff`, source `16:05:27:4c:93:c9`, EtherType `0x88b5`, and payload `hello from lab1`. The 14-byte Ethernet header and 14-byte payload are followed by 32 zero-padding bytes.